

You have stared at it: a pod stuck **Pending** that clearly should fit. The dashboard shows free chips. The job asks for fewer chips than are free. And still the scheduler refuses, over and over, with the same terse warning. This guide walks through five worked levels of that exact situation, decodes the messages the cluster gives you, shows *where the rules fight back*, and ends with debugging tactics and a small Python checker you can run before you ever apply a manifest.

The feeling this reproduces

“Free capacity that refuses to schedule your job.” The frustration is real and it has a precise cause: for many accelerator jobs, having *enough* chips is not the same as having the *right* chips in the *right shape*. That gap is the whole problem.

1 The four rules that make placement fight back

Every failure in the five levels traces back to one of four rules. None of them is visible on a capacity gauge, which is exactly why a half-empty cluster can still say no.

Rule	What it means	How it bites
Memory ceiling	Each accelerator holds a fixed amount of HBM.	A model larger than the ceiling is rejected before it starts.
All-or-nothing (gang)	A multi-chip training job needs every worker placed at once.	Land 3 of 4 chips and the whole job still waits.
Reclaimable (spot)	Cheap capacity is borrowed, not owned.	A guaranteed job reclaims it after a short preemption notice.
Grid topology	TPU chips are cabled to neighbours in a torus mesh.	A job needs a contiguous rectangle, not just N free chips.

The first three are intuitive once stated. The fourth — topology — is the one that produces the “should fit but doesn’t” feeling, so it gets the most attention below.

2 Decoding the error you keep seeing

Across all five levels, every stranded job reports the same line in the puzzle’s Events panel. It is written in the *style* of `kubectl describe` output — a deliberately simplified, illustrative string, not a verbatim Kubernetes message — so learning to read it here builds the same instinct you use on a real cluster.

Events (puzzle simulator) – illustrative, not literal cluster output

```
train-llm Warning FailedScheduling 0/10 accelerators assigned: no placement.
```

■ This exact string is illustrative

“0/10 accelerators assigned: no placement” is the simulator’s stylised wording. A real GKE scheduler counts **nodes**, not accelerators, and names the specific reason. The genuine message looks more like the block below — but the meaning you extract is identical.

Events – representative real kubectl describe output

```
Warning FailedScheduling default-scheduler
  0/5 nodes are available: 3 Insufficient google.com/tpu,
  2 node(s) didn't match Pod's node affinity/selector.
```

Read the meaning piece by piece — it carries across both the simulator and the real thing:

- **Warning FailedScheduling** — the scheduler ran, considered this pod, and could not place it. It is not a syntax error in your YAML; it is a placement failure.
- **0 of N available / assigned** — not one candidate satisfied the pod’s requirements. Crucially this means ‘no valid assignment exists right now’, *not* ‘every chip is busy’. The count near zero is what misleads you.
- **the reason clause** (‘no placement’ in the puzzle; ‘Insufficient google.com/tpu’ or an affinity mismatch on a real cluster) — the actual constraint that failed: topology, memory ceiling, gang-completeness, or a node selector.

■ The trap in “0 available”

The count tempts you to think “there aren’t enough free chips.” Often there are plenty. **0 available** means the constraints could not be satisfied, not that capacity was exhausted. Check *shape* and *ceilings* before you conclude you need a bigger cluster.

Documentation for the concepts behind this message:

- [About TPUs in GKE \(topology, slices, single- vs multi-host\)](#)
- [TPU v5e architecture and supported slice shapes \(1x1, 2x2, 2x4\)](#)
- [Gang scheduling for TPU multislice workloads](#)
- [Spot VMs and preemption](#)
- [Troubleshoot TPUs in GKE \(FailedScheduling and capacity\)](#)

3 The five levels, and where each one fights back

The levels share one host — `ct5lp-hightpu-8t`, a single TPU v5e machine of eight chips wired as a **2×4** grid — plus a set of GPUs in the later levels. Each level turns on one more rule. (Strictly, v5e chips are connected as a 2-D *torus*: edge chips wrap around to the opposite edge. For a single 8-chip host the plain-grid picture below is faithful enough; the wraparound only starts to matter on larger slices.)

Level	New rule introduced	The job that strands, and why
1	Contiguity (gang on a mesh)	train-llm needs a 2x2 block. Any four free chips is not enough; they must be adjacent.
2	Fragmentation	More serving jobs share the slice. Scatter them and no 2x2 rectangle survives for the gang.
3	Memory ceilings + spot	GPUs differ in size; a 64Gi model needs a GPU whose ceiling holds it. Spot job yields when a guaranteed job needs the card.
4	Larger slice (2x4 = 8 chips)	The gang now needs the WHOLE host as one rectangle. A single careless serving drop makes an 8-chip block impossible.
5	Everything at once	Protect a contiguous TPU rectangle, honour every GPU ceiling, and let spot ride the remainder.

The core picture: enough chips, wrong shape

Scattered: 5 free, NO 2x2 rectangle



Packed to one side: 2x2 gang fits

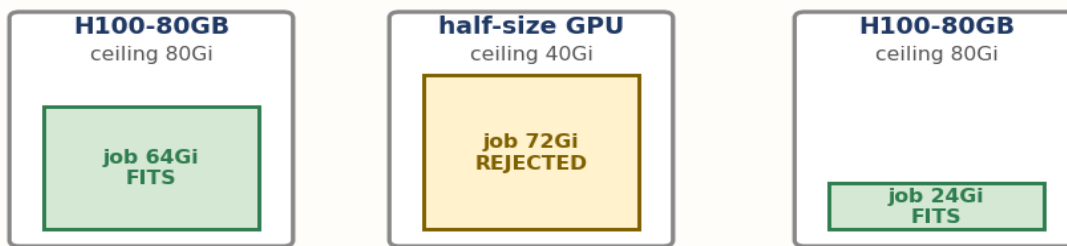


Left: three one-chip serving jobs (S) dropped carelessly leave five chips free — but no four of them form a rectangle, so the 2x2 training gang (T) cannot be placed. This is the Level 2 failure exactly. Right: seat the same three jobs against one edge and an unbroken 2x2 block survives. Same jobs, same host, opposite outcome — the only thing that changed is where the small jobs sit.

● **The mental model that fixes most of this**

Treat the mesh like parking a moving truck in a lot: don't scatter compact cars across every third space, or the truck (the gang job) never gets two adjacent spots. **Pack small jobs against an edge and keep the largest contiguous rectangle intact** for the job that needs it.

The other way it bites: the memory ceiling



Same job (72Gi) fits the 80Gi ceiling, but is turned away at the 40Gi door.

A GPU job carries a memory request. A device whose HBM ceiling is below that request turns the job away at the door — it never starts, exactly like Level 3's memory-heavy job that only the larger GPU can hold. Free cores on a too-small card do not help.

4 Debugging tips: turning “no placement” into a fix

When a pod is Pending and “should” fit, work through these in order. Each maps to one of the four rules, so the first one that matches usually explains the failure.

A quick triage checklist

- **Count vs. shape.** Does the job request a `tpuTopology` (e.g. 2x2, 2x4)? If so, free-chip count is irrelevant — you need a contiguous rectangle of that shape. Look for fragmentation first.
- **Check the ceiling.** For GPU jobs, compare the job's memory request against the target device's HBM. A 72Gi job cannot land on a 40Gi card no matter how idle it is.
- **Confirm gang-completeness.** A gang job places all-or-nothing. If only part of the slice is free, the whole job waits — partial capacity is not partial progress.
- **Spot vs. guaranteed.** If a spot job vanished mid-run, a guaranteed job likely reclaimed its capacity. That is expected behaviour, not a bug; give spot the leftover space, not the contested space.
- **Read the whole board, not the one pod.** The failing pod is rarely the cause. The *placement of everything else* is what left no room. Fix the layout, not the victim.

Concrete moves that resolve the five levels

Symptom in Events	Likely rule	Move that fixes it
0 available, plenty free	Topology / fragmentation	Re-seat small jobs to an edge; free a contiguous rectangle of the requested shape.
Memory-heavy job never lands	Memory ceiling	Target a device whose HBM $\geq$ request; move small jobs off the large GPU.
Gang job waits with 3/4 free	All-or-nothing	Free the last adjacent chip; never leave a gang partially placed.
Spot job stopped mid-run	Reclaimable capacity	Expected on preemption; reschedule spot onto genuinely spare chips.
8-chip gang can't fit	Whole-host rectangle	Keep the entire 2x4 clear for the gang; place serving jobs only after.

■ Useful commands when you hit this live

`kubectl describe pod <name>` shows the Events block quoted above. `kubectl get pods -o wide` shows what already sits where. For TPUs, inspect the node labels `cloud.google.com/gke-tpu-topology` and `-accelerator` to confirm the slice shape you actually have.

5 Can you check it automatically? Yes — in Python

“Getting the right jobs onto the right chips is a packing problem,” and packing problems can be checked mechanically before you apply a manifest. The two checks worth automating are the two that cause every level's failure:

- **Rectangle check** — given what serving jobs already occupy, does a contiguous block of the requested topology still exist on the mesh?
- **Ceiling check** — can each GPU job land on a device whose HBM ceiling is at least its memory request?

Below is the heart of a small checker (the full script ships alongside this PDF as `tpu_placement_checker.py`). It models the host as a grid, marks the chips serving jobs hold, and searches for a free rectangle — the same search a scheduler does, made visible.

■ What the script does, in plain English

Think of the host as an egg carton of chip slots. The script **colours in** the slots that small serving jobs already sit on, then **slides a rectangle** of the size your training job needs (say 2×2) across every remaining position to see if it fits in one unbroken block. If it finds a spot, it prints OK and the coordinates; if it can't — even when there are enough empty slots overall — it prints the reason the job would be stuck. It also does a second, simpler check for GPUs: does the job's memory request fit under the card's ceiling? You run it before applying a manifest, so you catch a doomed layout in one second instead of after a failed deploy. No cluster or cloud account is needed — it's a self-contained model you can read and edit.

tpu_placement_checker.py (excerpt)

```
def find_rectangle(mesh, h, w):
    # Return cells of the first free h x w (or w x h) block, else None.
    for (rr, cc) in ((h, w), (w, h)):          # try both orientations
        for r0 in range(mesh.rows - rr + 1):
            for c0 in range(mesh.cols - cc + 1):
                block = [(r0+dr, c0+dc)
                        for dr in range(rr) for dc in range(cc)]
                if all(mesh.occupied[rc] is None for rc in block):
                    return block                # a contiguous slice fits
    return None                                # -> FailedScheduling: no placement

def check_gang_job(mesh, topology, name):
    h, w = parse_topology(topology)            # '2x2' -> (2, 2)
    need, have = h * w, mesh.free_count()
    block = find_rectangle(mesh, h, w)
    if block is None and have >= need:
        return f'{have} free chips but no {h}x{w} rectangle -- fragmented'
    return 'OK' if block else f'only {have} free, need {need}'
```

Running the shipped script prints the failure and the fix side by side, mirroring Level 2:

```
$ python3 tpu_placement_checker.py
Mesh after serving jobs (scattered):
S . S .
. S . .
Free chips: 5 of 8
FAIL train-llm: 5 free chips but no contiguous 2x2 rectangle -- fragmented
-> (simulator) Warning FailedScheduling: no placement.
-----
Mesh after serving jobs (packed to one side):
. . S S
. . . S
OK train-llm takes contiguous rectangle [(0,0),(0,1),(1,0),(1,1)]
```

● How to use this in review

Wire the rectangle and ceiling checks into a pre-submit step: feed it your intended layout and have it reject any plan that leaves no room for the gang job or overflows a GPU ceiling. It turns a “stared-at-it-for-an-afternoon” failure into a one-second automated no.

Where to take it next

- Extend `parse_topology` to real 3-D meshes for v4/v5p/Ironwood hosts, where the block search becomes 3-dimensional. Note that 2x2x1 is only a minimal example — production v5p slices are far larger (e.g. 16x20x28).
- Model **torus wraparound**: on v5e/v6e and the 3-D generations, edge chips connect to the opposite edge, so a block spanning an edge can still be contiguous. A plain grid search (like the one above) misses those valid placements on larger slices.
- Add a spot-reclaim pass: when a guaranteed job needs a card a spot job holds, free it and re-check.
- Feed it live state from `kubectl get pods -o json` so it checks the actual cluster, not a hand-built model.

In one sentence

A pod that “should fit” usually fails because **enough chips is not the same as the right chips in the right shape under the right ceiling** — and once you can read the “no placement” message and check the packing in Python, the frustration becomes a solvable, automatable layout problem.